

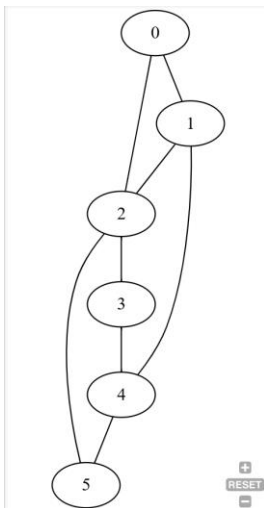
Visite grafi

Codice:

ho implementato, come richiesto, le classi `lifo.hpp` e `fifo.hpp`.

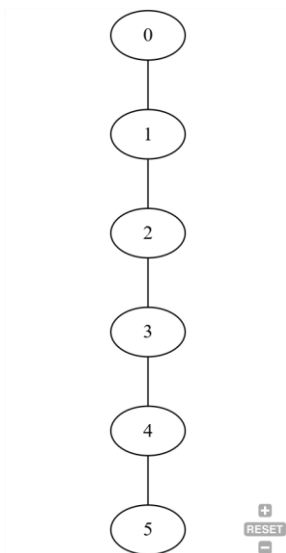
`main.cpp`

nel main ho costruito un grafo `g1` non pesato:

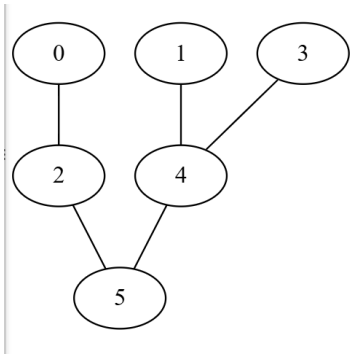


Ho stampato gli alberi di visita per:

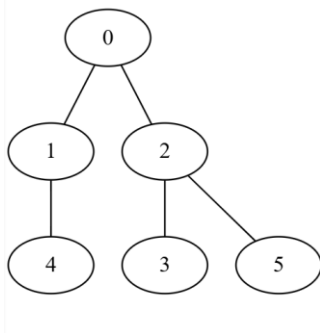
dfs ricorsiva (partendo da 0):



Dfs iterativa (partendo da 0):



Bfs



Successivamente, ho costruito un grafo pesato e ho calcolato i percorsi più “brevi” per arrivare a ciascun nodo mediante l’algoritmo di Dijkstra, sempre partendo dal nodo 0. Ho controllato a mano la correttezza del risultato.

visit_again.hpp

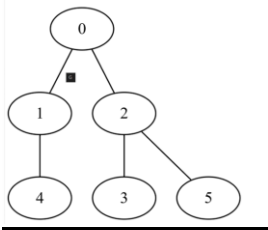
l’algoritmo di visita del grafo che sfrutta queste strutture è salvato in visiti_again.hpp

Qui ho costruito un contenitore (lifo o fifo) di archi, con una classe “dir_edge” definita ad hoc. Questo mi ha permesso di salvare i nodi di ciascun arco con l’ordine desiderato (al contrario di “unidir_edge”).

Ho costruito una lista di dir_ege chiamata “visit”, dove salvo gli archi che comporranno l’albero della visita. Ho costruito un unordered set chiamato “visitati” che, ad ogni iterazione del while, mi consenta di verificare di non aver già visitato il nodo a cui “punta” l’arco.

Per la costruzione del grafo ho sfruttato un costruttore definito nella classe “grafo”, che riceve una mappa di set vuoti (in cui le chiavi della mappa sono i nodi del grafo) e un vettore di unidir_edge. Quindi, ho convertito la lista di dir_edge in un vettore di unidir_edge e ho estratto i nodi dal grafo. In quest’ultima parte ho utilizzato l’ai per controllare l’uso degli iteratori.

Inizialmente avevo implementato l'algoritmo in modo sbagliato (codice: visit.hpp), poiché per ogni vicino di "e" prima si verifica se è stato visitato o meno, poi si mette nell'albero delle visite e solo alla fine si aggiunge al contenitore (lifo o fifo). Pertanto, nella ricerca dfs il grafo non veniva esplorato in profondità, come si vede dall'albero di ricerca che ne risultava:



L'idea di inserire gli entrambi i nodi (precedente e successivo) nel contenitore, mi è stata suggerita dall'ai. Ho scelto di implementarla costruendo una classe `dir_edge` di archi "diretti".

dfs_recursive

nel file `dfs_recursive_final.hpp` la funzione `dfs_rec_final` crea una mappa di set chiamata "visit", che salverà i nodi dell'albero della visita.

grafo_pesato.hpp

Ho costruito la classe `grafo_pesato` per implementare l'algoritmo di dijkstra. È costruita come la classe `grafo` della scorsa esercitazione, ma, invece che salvare gli archi in un vettore, li ho salvati come chiavi di una mappa, i cui elementi sono i pesi.

Ho aggiunto la funzione `peso`, che, dati due nodi, restituisce il peso dell'arco che li congiunge. Se l'arco non esiste, si solleva un'eccezione.

coppie.hpp

ho costruito questa piccola classe per assegnare, nell'algoritmo di Dijkstra, una distanza ad ogni nodo. Ho implementato gli operatori di confronto (`>` e `==`) per confrontare le distanze.

Ho scelto di stabilire che una coppia è `>` di un'altra se ha distanza minore dal nodo "start", in modo che viene messa nel posto "first out" della priority queue.

Dijkstra

Per ogni nodo costruisco una lista che conterrà i nodi del percorso più breve per arrivarci.

